Serverless Student Performance Analytics System

Objective

Build a **serverless data processing system** using **AWS S3, AWS Lambda, DynamoDB, and Python** to analyze student academic performance data.

The system should automatically process uploaded datasets, store the records in DynamoDB, and support CRUD operations and analytical queries.

Dataset

You are provided with a dataset containing the following columns:

DataSet:https://www.kaggle.com/datasets/nabeelqureshitiii/student-performance-dataset/data

student_id
weekly_self_study_hours
attendance_percentage
class_participation
total_score
grade

Example Data


```
[
 {
   "student_id": "ST001",
   "weekly_self_study_hours": 12,
   "attendance_percentage": 92,
   "class_participation": 8,
   "total_score": 88,
   "grade": "B"
 },
 {
   "student_id": "ST002",
   "weekly_self_study_hours": 15,
   "attendance_percentage": 96,
```

```
  "class_participation": 9,
  "total_score": 93,
  "grade": "A"
 }
]
```


Scenario

A university collects **weekly student performance reports** containing information about study hours, attendance, participation, and scores.

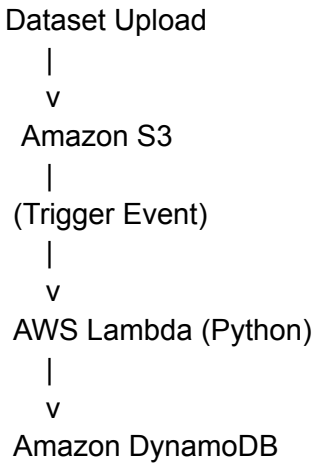
Faculty members upload these reports to an **Amazon S3 bucket**.

Whenever a file is uploaded:

1. The system must automatically process the file.
2. Extract the student records.
3. Store them in **Amazon DynamoDB**.
4. Allow queries and analysis of student performance.

The system must be implemented using **serverless architecture**.

System Architecture



Requirements

Part 1 — DynamoDB Table Design

Create a DynamoDB table named:

student_performance

Attributes

student_id
weekly_self_study_hours
attendance_percentage
class_participation
total_score
grade
performance_category

Primary Key

Partition Key: student_id

Part 2 — Performance Category Calculation

Your Lambda function must calculate a new attribute called:

performance_category

Rules

Total Score	Category
>= 90	Excellent
75 – 89	Good
60 – 74	Average
< 60	Poor

Part 3 — S3 Upload

Create an S3 bucket.

Example:

student-performance-datasets

Upload JSON files containing student records.

Configure **S3 event notification** to trigger a Lambda function whenever a file is uploaded.

Part 4 — Lambda Processing

Write a **Python Lambda function** that:

1. Reads the uploaded file from S3
2. Parses the JSON data
3. Calculates `performance_category`
4. Inserts records into DynamoDB

The function should handle multiple records in a single file.

Part 5 — CRUD Operations

Write Python scripts to perform the following operations using DynamoDB.

Create

Insert a new student record.

Read

Retrieve student performance data using:

student_id

Update

Update any of the following fields:

weekly_self_study_hours
attendance_percentage
class_participation
total_score
grade

Delete

Delete a student record.

Part 6 — Query Operations

Implement the following queries.

Query 1

Retrieve students with:

attendance_percentage > 90

Query 2

Retrieve students who studied more than:

weekly_self_study_hours > 10

Query 3

Retrieve students in the **Excellent performance category**.

Part 7 — Secondary Index

Create a **Global Secondary Index (GSI)**.

Example:

Partition Key: grade
Sort Key: total_score

Use this index to retrieve:

- Top students in grade A
- Students with highest scores

Part 8 — Data Import

Upload a dataset containing **at least 200 student records** to S3.

Verify that the Lambda function processes the data and stores it in DynamoDB.

Part 9 — Data Export

Export student performance records from DynamoDB into a file.

Supported formats:

JSON
CSV

Upload the exported file to S3.

Advanced Tasks

1 Student Risk Detection

Add a new field:

at_risk

Mark students as **True** if:

attendance_percentage < 60
OR
total_score < 50

2 Top Performers

Generate a leaderboard of:

Top 10 students by total_score

3 Batch Data Processing

If a file contains **more than 500 records**, use **batch write operations** to insert records efficiently.

4 Error Handling

Handle the following cases:

- Invalid JSON file
- Missing fields
- Duplicate student records

Deliverables

Candidates must submit:

1. DynamoDB table schema
2. Lambda function code
3. Python scripts for CRUD operations
4. Sample dataset file
5. Exported data file
6. Screenshots of AWS resources
7. Architecture diagram